Ders 2 Genişletilmiş Notu: Öz Denetimli Öğrenme, Üretici Modeller ve RAG

Ders 2 Genişletilmiş Notu: Öz Denetimli Öğrenme, Üretici Modeller ve RAG

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Giriş: Öğrenme Türleri ve Etiketli Veri Sorunu
- 2. Öz Denetimli Öğrenme Nedir?
- 3. Otokodlayıcılar ve Gizli Uzay
- 4. Varyasyonel Otokodlayıcılar ve Difüzyon Modelleri
- 5. Tamamlayıcı Modeller
- 6. Karşısal Öğrenme
- 7. İnsan Ödülü Olmadan Pekiştirmeli Öğrenme
- 8. Öz Denetimli Öğrenmenin Toplu Görünümü ve Ara Değerlendirme
- 9. Üretici Modeller
- 10. Transformer ve Öz Dikkat
- 11. Büyük Dil Modelleri
- 12. Halüsinasyon, İnce Ayar ve RAG
- 13. Colab Uygulaması: Otokodlayıcı ile MNIST Sınıflandırma
- Örnek Sorular
- Dersin Toparlaması

1. Giriş: Öğrenme Türleri ve Etiketli Veri Sorunu

Ders online yapıldı ve makine öğrenmesi konularının son dersiydi. Hoca iki temel konu olduğunu söyledi: öz denetimli öğrenme ve üretici modeller. Bu iki konuyu “Fatih Hocamızın” slaytlarından anlattı. Bunlara ek olarak, ileride projelerde kullanılabilecek yeni teknolojiler için kendi hazırladığı ek slaytları da gösterdi.

Hoca önce makine öğrenmesinin baştan beri gördüğümüz parçalarını hatırlattı: eğiticili öğrenme (supervised learning), eğiticisiz öğrenme (unsupervised learning) ve pekiştirmeli öğrenme (reinforcement learning). Yaklaşık on yıl önce bu dersi alan bir öğrenciye makine öğrenmesi bu üç parça olarak anlatılırdı. O zamanki çalışmalar daha çok belirli (spesifik) problemleri çözmeye yönelikti. Bugünkü gibi dil modelleri ve genel amaçlı araçlar pek yoktu. Hedef örneğin yalnız yüz tanıma, belli bir alana ait metinleri sınıflandırma ya da bunlarla ilgili kümeleme problemleriydi. Etiketli veri yoksa kümeleme yapılıyordu. Pekiştirmeli öğrenmede de belli oyunların ya da problemlerin çözümü yapılıyordu.

Bu dersin asıl konusu, eğiticili öğrenmenin bir açmazıdır: yeterli etiketli veri bulmak.

1.1 Banka Dekontu Örneği

Hoca kendi tecrübesinden bir örnek verdi. Bir bankada dekontlar vardır ve bunların insanlar tarafından sınıflandırılması gerekir. Dekontun üzerinde makine yazısı da vardır, el yazısı da.

Dekont A4 boyutunda olmayan dikdörtgen bir kağıttır ve sınıflandırma için odaklanılması gereken belli bir bölge vardır. Diyelim ki sol alt köşede el yazısıyla "fatura ödemesi" ya da "kredi ödemesi" gibi bir şey yazıyor. Fatura ise bir kenara, kredi ödemesi ise başka bir kenara konur.

Bu işi önce insanlar yapar. Bir süre sonra banka bu işi makineye yaptırmak ister. Bunun için elinde yeterince sınıflandırılmış veri vardır. Problem temelde bir OCR (optik karakter tanıma) problemidir: Dekont taranır, ilgili bölgedeki metne odaklanılır, OCR yapılır ve sonuçta faturayla ilgili bir anahtar kelime varsa "fatura ödemesi" sınıfına, yoksa "kredi" ya da başka bir sınıfa atanır.

Hoca bu problemin küçük ölçekli olduğunu vurguladı. Veri sayısı banka ölçeğinde fazla olsa da sınıf sayısı iki-üç tanedir, bin çeşit belge yoktur. Örneğin 5 sınıf olsun ve her sınıftan 5.000 örnek toplansın. Bu durumda bankada $5 \times 5000 = 25.000$ etiketli örnek vardır. Bu veriyle, derin öğrenmeye bile girmeden, OCR'dan çıkan metin KNN, Naive Bayes, karar ağacı, SVM ya da yapay sinir ağı gibi yöntemlerle sınıflandırılabilir.

1.2 Etiketli Verinin Yetmediği Problemler

Zamanla problemler yalnız alana özgü problemlerden oluşmamaya başladı. Bir kısmı genel amaçlıdır ve her türden soruya yanıt vermesi beklenir. Bir kısmında ise belki yalnız 5-6 sınıf vardır ama yeterli etiketli veriye ulaşmak zordur. Hoca bu ikinci durumun çok önemli olduğunu söyledi.

Örnek olarak proje öğrencileriyle yaptığı müzik sınıflandırma çalışmasını anlattı. Batı müziği için 8.000 ve 24.000 örnekli hazır veri setleri vardır ve bunlarla yaklaşık 20 müzik türü sınıflandırılır. Bu çalışmayı Türk sanat müziği makamlarına uyarlamak istediler: Muhayyer Kürdi, Kürdilihicazkar gibi yaklaşık 20 makam. Fakat bu konuda yeterli etiketli veri yoktur. 20-30 örnekle yapılmaya çalışıldığında yeterli başarıya ulaşılamaz. Hoca buradan "bundan bir iş olmaz" sonucunun çıkmadığını, olsaydı bugünkü ChatGPT ve benzeri araçların bu noktaya gelemeyeceğini söyledi.

Etiketli veriye ulaşmanın klasik yolu, alan uzmanlarını istihdam edip veriyi onlara etiketletmektir, yani para harcamaktır. Ancak para olsa bile her problemde yeterli etiketleme yapılamaz.

> **Hocanın vurgusu:** Makine öğrenmesinde en büyük darboğaz biziz, yani insan. İnsan olarak yeterli etiketli veriyi sağlayamıyorsak makine öğrenemez.

Buradan şu soru doğar: Az sayıda etiketli veriden yola çıkarak etiketleme işini de makinelere yaptırabilir miyiz? Hoca durumu kademeli olarak özetledi:

- Veri var, etiket var: klasik eğiticili öğrenme.
- Etiket yok: en kötü ihtimalle kümeleme (clustering).
- Veri var, etiket az ya da yok: yarı denetimli (semi-supervised) ve öz denetimli (self-supervised) öğrenme.
- Veri de yok: Örneğin bir alanda yalnız 50 veri var ama 50.000 veriye ihtiyaç var. Bu durumda üretici modeller (generative models) devreye girer. Bu, dersin ikinci parçasıdır.

2. Öz Denetimli Öğrenme Nedir?

Hoca öz denetimli öğrenmeyi yarı denetimli öğrenmenin bir alt konusu olarak tanıttı. İçinde farklı stratejiler vardır: otokodlayıcılar, tamamlama yöntemleri, karşısal öğrenme (contrastive learning) ve insan ödülü olmadan pekiştirmeli öğrenme. Hepsinde hedef, ekstra insan emeğine ihtiyaç duymadan öğrenmektir. İşin sonunda makine kendi eğitim kümelerini üretir. Bu sayede günümüzün dil modelleri dahil birçok problem daha verimli çözülebilir hale gelir.

Hoca burada yalnız veri değil, hata da üretildiğini söyledi. Makine hatanın nasıl üretildiğini öğrenirse, o hatayı nasıl gidereceğini de öğrenir. Otokodlayıcılar bu mantıkla çalışır.

Hoca, bu konuları 15-20 yıl önce kendi lisans öğrenciliği zamanında bir hoca anlatsaydı "hayali şeyler anlatıyor" denebileceğini, ama bugün bunların meyvelerinin karşımızda olduğunu söyledi. Öz denetimli öğrenme, son dönem başarılarının en önemli bileşenlerindendir. Hem ucuz hem hızlı biçimde çok büyük miktarda eğitim örneği üretilebilir. Ayrıca insan davranışlarından, insan hatalarından ve insanlar arası farklardan bağımsızlık ve çeşitlilik sağlanır.

2.1 Bebek Benzetmesi

Hoca kendi hazırladığı slayttan bir benzetme yaptı. Dünyaya gelen bir bebeğe hiçbir şey etiketlenmez. Ona binlerce kez "bu kedidir" denmez. Ama bebek bir süre sonra kedileri ve kediye benzeyen şeyleri yavaş yavaş tanır. Kendi deneyimi ve tahmini üzerinden, gelen geri bildirimle düzelterek öğrenir. Öz denetimli öğrenme de tam olarak bunu yapar: Kendi öğrenme sinyalini kendisi üretir.

Karşılaştırma şöyledir:

- **Denetimli öğrenme:** Etiket gerekir. Örneğin bir milyon kedi fotoğrafının her birine "bu kedidir" etiketi verilmelidir.
- **Denetimsiz öğrenme:** Etiket yoktur, kümeleme yapılır.
- **Öz denetimli öğrenme:** Model kendi etiketini veriden üretir.

Hocanın metin örneği: Bir cümlenin olumlu mu olumsuz mu olduğunu insanlara etiketletirseniz denetimli öğrenme yaparsınız. Ama cümlenin son kelimesini silip modelden onu tahmin etmesini isterseniz öz denetimli öğrenme yaparsınız. Çünkü bu bilgi eğitim verisinin içinde zaten vardır, yalnız gizlenmiştir.

> **Hocanın vurgusu:** Cevap verinin kendisinde vardır. Sadece belli bir parçası maskelenmiş, gizlenmiş, silinmiş ya da bozulmuştur. Model bu parçayı tahmin eder.

Örnek: "İstanbul'un tarihi … güzeldir." Model yeterince öğrendiğinde bu boşluğu kendisi doldurabilir. Cevap metnin içindedir.

2.2 Ölçek Etkisi ve Denetimli Öğrenmenin Sınırları

Slayttaki bir örneğe göre bir milyon örnek etiketletilirse bir problem orta düzeyde, yaklaşık %81 başarıyla çözülebilir. Hoca bir milyon etiketin günümüz için çok fazla olmadığını hesapla gösterdi. Bin kişi istihdam edilir ve her biri bin etiket verirse $1000 \times 1000 = 1.000.000$ etiket elde edilir. Bu belki 3 günde rahatça biter. Bin değil de yüz işçi olsa belki 3 değil 10 günde biter, ama biter.

Örnek sayısı trilyon mertebesine çıkınca etiketleme insan kapasitesinin üzerine çıkar. Slayta göre ImageNet'teki 14 milyon görüntüyü etiketlemek için yılda 25 bin kişi tam zamanlı çalışmalıdır. Hoca bunu denetimli öğrenmenin sınırı olarak adlandırdı. Ayrıca slayta göre bugünkü internetin %1'i bile denetimli öğrenme için etiketlenemez.

> **Ders sonrası not:** Hoca GPT-3 için "175 bin parametre" dedi; doğrusu 175 milyar parametredir.

3. Otokodlayıcılar ve Gizli Uzay

Otokodlayıcı (autoencoder) eskiden bir yapay sinir ağı yapısı olarak önerilmiştir. Hedef, veriyi sıkıştırmak ve verinin önemli kısımlarını almaktır.

Hoca ekrandaki slaytı örnek verdi. Slaytın ekran görüntüsünü alırsanız, bilgi içeren bölgeler de, beyaz boşluklar ve düz mavi alanlar da görüntünün içindedir. Otokodlayıcı bu veriyi en iyi temsil edecek bir vektör uzayına indirger. Bu uzaya gizli uzay (latent space) denir.

Neden gerekli? 1000×1000 piksellik bir görüntüde $1.000.000$ değer vardır. RGB düşünülürse her piksel üç renge karşılık geldiği için $3.000.000$ değer olur. Bu görüntüyü bir veri aktarımında kullanacaksanız ya da üzerinde OCR yapacaksanız, girdisi 1 ya da 3 milyon olan bir özellik vektörüyle çalışmanız gerekir. Hoca bunun neredeyse problemi çözememek demek olduğunu söyledi. Veriyi en iyi temsil eden bir gizli uzay bulunursa, yani veri en uygun şekilde sıkıştırılırsa, hem iletim hem sınıflandırma iyileşir.

Yapı şöyledir:

- Girdi x, kodlayıcıdan (encoder) geçer ve z gizli vektörü elde edilir.
- z, çözücüden (decoder) geçer ve yeniden oluşturulmuş $\hat{x}$ elde edilir.
- Eğitimde x ile $\hat{x}$ arasındaki fark en aza indirilir.

Hoca ses ve görüntü gibi verilerde küçük bir hatanın kabul edilebileceğini, ama metin gibi verilerde daha küçük hatalar gerektiğini ekledi.

Hoca otokodlayıcıyı bir kağıdı buruşturup ya da katlayıp olabildiğince küçültmeye, sonra geri açmaya benzetti ve geri açınca orijinalden bir şey kaybolmadığını söyledi. Makinede bunu yapmak için verinin özünü kaybetmeyecek şekilde sıkıştırmak gerekir.

> **Ders sonrası not:** Kağıt benzetmesi tam doğru değildir. Otokodlayıcı kayıplı sıkıştırma yapar; amaç kaybı sıfırlamak değil, en aza indirmektir. Hoca da kod üzerinde x ile $\widehat{x}$'in birebir aynı olmasını beklemediğimizi söyledi.

3.1 MNIST Örneği

Hoca MNIST veri setini gösterdi: 28×28 piksellik el yazısı rakam görüntüleri. Bir görüntüde $28 \times 28 = 784$ piksel vardır. Bu 784 piksel doğrudan özellik olarak alınıp eğiticili öğrenme yapılabilir. Ancak 784 özellik hem çok fazladır hem de içinde çok gereksiz veri vardır. Rakamın arkasında kocaman siyah bölgeler vardır, yani matris büyük ölçüde sıfırlarla doludur. Vektöre çevrilse de bir sürü sıfır olur. Bu hem çözümü zorlaştırır hem de başarımı düşürebilir.

Otokodlayıcıyla bu veri 32 boyutlu bir uzaya indirgenebilir: $784 \to 32$. Bu 32 boyutlu vektörler üzerinde KNN gibi bir algoritmayla sınıflandırma yapılabilir. Hoca bunu canlı denemek için bir yapay zeka aracına şu isteği yazdı: "MNIST verisine önce otokodlayıcı uygula, sonra 32 uzunluklu vektörlerle k-katlı çapraz doğrulama (k-fold cross-validation) yaparak KNN, SVM ve MLP ile sınıflandır." İlk araç o an yanıt vermedi, hoca başka araçlara geçti. Sonuçlar dersin sonunda gösterildi (Bölüm 13).

Bu indirgeme sayesinde hem sınıflandırma daha performanslı yapılabilir hem de yeni veri üretme imkanı doğar. Elde edilen vektör verinin özünü taşıyan bir öz temsildir.

3.2 Kodun Yapısı

Hoca yapay zeka aracının ürettiği kodu açıkladı. Otokodlayıcı bir kodlayıcı ve bir çözücüden oluşur:

- **Kodlayıcı:** $784 \to 128 \to 64 \to 32$.
- **Çözücü:** 32'den tekrar 784'e açar ve $\widehat{x}$'i üretir.

Hoca bunu bir yapay sinir ağı olarak tahtaya çizdi: 784 girdili bir katman, sonra 128, sonra 64, sonra 32 nöronlu katmanlar. Öğrenilen şey aradaki ağırlıklardır. Öğrenme, kodlamayı ve çözmeyi birlikte kapsar. Hata ölçütü ortalama kare hatadır (mean squared error, MSE). Otokodlayıcı eğitildikten sonra her görüntü için 32'lik bir vektör elde edilir. Artık 784'lük vektörle işimiz kalmaz. Bu vektörlerle k-katlı çapraz doğrulama yapılarak KNN, SVM ve MLP ile sınıflandırma yapılır. Karşılaştırma için 784 boyutlu ham veriyle de aynı deney yapılmalıdır.

4. Varyasyonel Otokodlayıcılar ve Difüzyon Modelleri

4.1 Klasik Otokodlayıcı ile VAE Farkı

Klasik otokodlayıcı veriyi keskin biçimde sıkıştırmayı öğrenir ama yeni veri üretmede zayıftır. Bunun için varyasyonel otokodlayıcılar (variational autoencoder, VAE) vardır.

Hoca farkı şöyle anlattı: Klasik otokodlayıcı 784 boyutlu veriyi 32 boyuta indirdiğinde, bu 32 boyutlu uzayda tek bir nokta elde edilir. 3 boyuta indirgense x, y, z koordinat sisteminde bir nokta olurdu. VAE'de ise veri, gizli uzayda bir bölgeye karşılık gelir. Bu bölge bir ortalama ve standart sapmayla tanımlanan bir dağılımdır. "Varyasyon" adı buradan gelir. Gizli temsil tek bir nokta değil, bir noktalar kümesi, yani bir bölgedir.

Bu bölgeden rastgele bir nokta alınıp çözücüye verildiğinde gerçekçi yeni veriler üretilebilir.

> **Hocanın vurgusu:** Klasik otokodlayıcı veriyi gizli uzayda tek bir noktaya, VAE ise bir bölgeye (dağılıma) eşler. Bu yüzden VAE yeni veri üretebilir.

4.2 Difüzyon Modelleri

Difüzyon modellerinde elimizde örnek bir veri, örneğin bir görüntü vardır. Bu veriye ileri yönde (forward) adım adım bozucu etki, yani gürültü eklenir. Bozma bilinçli yapılır ve sonunda görüntü saf gürültüye dönüşür. Sistem bu gürültünün nasıl eklendiğini öğrenir. Sonra bu yolu

tersine (reverse) çevirerek gürültüyü adım adım temizler ve veriyi geri bulmaya çalışır. DALL-E ve Stable Diffusion bu tür modellere örnektir. Hoca öğrencilere bunları kullanıp kullanmadıklarını sordu.

Öz denetimli öğrenmeyle bağlantı şudur: Model her adımda ne kadar gürültü eklendiğini öğrenir. Cevap verinin içinde olduğu için insan etiketine gerek yoktur.

Hoca VAE ve difüzyon modellerinin otokodlayıcıların geliştirilmiş biçimleri olduğunu, ikisinde de bir kodlayıcı mantığı bulunduğunu söyledi. Ayrıntılarına ve matematiğine girmedi, ilgilenenlerin slayttaki kaynaklara bakmasını önerdi.

> **Ders sonrası not:** Hoca bir yerde difüzyon modelleri için "bunların hepsi varyasyonel otokodlayıcılar" dedi. Difüzyon modelleri VAE ile akraba bir üretici model ailesidir ama VAE'nin kendisi değildir; ikisi ayrı model türleridir.

5. Tamamlayıcı Modeller

Tamamlayıcılar da bir öz denetimli öğrenme stratejisidir. Verinin içindeki parçalara odaklanırlar. Veri bir metin, görüntü ya da ses olabilir. Yaklaşımlar şunlardır:

- **Silinen parçayı tahmin etme:** Verinin bir kısmı silinir ve model onu tahmin eder.
- **Sonraki parçayı tahmin etme:** Sonraki kelime ya da cümle tahmin edilir. GPT ve Vision Transformer bunu yapar.
- **Eksik parçayı tahmin etme:** Metindeki boşlukları dolduran BERT modeli. Hoca BERT'i günümüz dil modellerinin atalarından biri olarak andı. Görüntülerde bunun karşılığı inpainting'dir.
- **Çevreyi tahmin etme:** Bulunduğun yerin etrafını tahmin etme. Word2Vec eski bir yöntemdir ama bazen işe yarar.

5.1 Word2Vec ve Kedi Örneği

Hoca öğrencilere "kediyle alakalı kavramlar ne olabilir?" diye sordu. Yanıt gelmeyince kendisi örnek verdi: Kedi ile "mama" yakındır (kedi maması). Kediyle alakası olmayan "bisiklet tekerleği" ise daha uzaktadır. Word2Vec kelimeyi vektöre dönüştüren bir yapıdır. Benzer kullanılan kelimeleri vektör uzayında yakın, benzemeyenleri uzak tutar. Bu fikir görüntü ve başka veriler için de düşünülebilir.

5.2 Otoregresif ve Maskeli Modeller

Otoregresif modeller geçmişe bakarak bir sonraki kelimeyi ya da pikseli tahmin eder. Örnek: "İstanbul'un tarihi …" cümlesinde sonraki kelime "yarımadası" olabilir. Bu kısmı otoregresif model tahmin eder.

Eksik parça tahmini ise örneğin "İstanbul [MASK] tarihi" gibi bir cümlede maskeli kelimeyi tahmin etmektir. Model "büyük", "görkemli", "muhteşem" gibi tahminler üretir.

Çevre tahmini: Word2Vec'e "kedi" verildiğinde "tatlı", "güzel", "oyuncu" gibi kelimeleri yakınına alır, alakasızları uzak tutar.

Hoca bunun günümüz dil modelleri için önemini vurguladı. Bu modeller yalnız kelimenin sayısal haliyle ilgilenmez. Doğru tahmin için kelimenin anlamına, hatta cümle içindeki anlamına, yani bağlamına bakmak gerekir. Bağlamı anlamak için de dil bilgisinin kavranmış olması gerekir. GPT ve BERT gibi modeller milyarlarca cümle gördükten sonra, etiket olmadan, anlamış gibi davranabilir. Bu kadar veriyi etiketlemek neredeyse imkansızdır.

GPT'nin temelinde bu otoregresif fikir vardır: bir kelime tamamlama oyunu. Başlangıçta insanların etiketlediği bir birikim vardır, ama bir süre sonra ölçek tamamen makine tarafından yürür. Hoca slayta göre GPT-1'de 117 milyon parametre olduğunu, GPT-4'te ise 1 trilyon parametre olduğunun söylendiğini aktardı. Hepsi aynı fikre dayanır: bir sonraki kelimeyi tahmin etmek. Örnek: "Yapay …" verilince "zeka", "Yapay zekaya girdin …" verilince "çok" gibi.

> **Ders sonrası not:** GPT-4'ün parametre sayısı OpenAI tarafından açıklanmadı. Derste geçen "1 trilyon" büyük olasılıkla bir tahmindir.

5.3 GPT ile BERT Karşılaştırması

GPT bir kelimenin sonrasını tahmin eder, soldan sağa bakar. BERT maskeli dil modelidir ve iki yönlü çalışır, hem sola hem sağa bakar. Örnek: "Ankara Türkiye'nin başkentidir." BERT "Türkiye'nin" kelimesini değerlendirirken yalnız öncesine bakmaz, sonrasındaki kelimelere de bakar. BERT eğitiminde kelimelerin %15'i maskelenmiştir.

	GPT	BERT
Yön	Soldan sağa	İki yönlü
Görev	Sonraki kelimeyi tahmin	Eksik (maskeli) kelimeyi tahmin
Güçlü olduğu alan	Üretim	Anlama ve sınıflandırma
Kullanım	ChatGPT'nin atası	Arama motorları

Hoca Word2Vec'e tekrar dönerek "kral", "erkek", "kadın", "kraliçe" ilişkisini andı: Bu kelimeler arasındaki vektör ilişkileri birbirine benzetilebilir.

5.4 Diğer Tamamlama Yaklaşımları: ELECTRA, BART, NSP

ELECTRA: Kelimeyi maskelemek yerine değiştirir. Bir üretici (generator) model bir kelimeyi değiştirir, bir ayırıcı (discriminator) model de her kelimenin orijinal mi değiştirilmiş mi olduğunu bulur. Slayttaki örnek: Orijinal metin "The chef cooked the meal", değiştirilmiş metin "The chef ate the meal". Ayırıcı "ate" kelimesinin değiştirildiğini bulmayı öğrenir. Hoca sözlü olarak, "kedi" yerine "köpek" yazılmışsa modelin bunu yakalayacağını söyledi.

Parçaların yerini karıştırıp düzeltme: Metnin parçalarının yerleri karıştırılır ve model doğru sırayı bulmayı öğrenir.

BART: Metni birden çok şekilde bozar: token silme, yer değiştirme, döndürme. Görüntüde de döndürme yapılabilir. Model orijinal metni yeniden oluşturmayı öğrenir. Özetleme ve çeviri için iyidir, örneğin İngilizceden Fransızcaya çeviri yapan araçlar bunu kullanır.

Sıra tahmini: BERT'teki NSP (next sentence prediction) gibi. İki parçanın ardışık ve doğru sırada olup olmadığı evet/hayır şeklinde tahmin edilir.

Görüntü tamamlama ve çözünürlük artırma: Görüntünün bir bölgesi silinir ve model onu doldurmaya çalışır.

6. Karşısal Öğrenme

Karşısal öğrenme (contrastive learning), verileri zaman ve mekandaki birlikteliğe göre birbirine yaklaştıran ya da uzaklaştıran bir öğrenme biçimidir.

Hocanın örneği bir köpektir. Köpeğin vücudundan farklı parçalar alınır. Parçalar birbiriyle alakalıysa (aynı köpeğe aitse) birbirine yaklaştırılır. Başka bir nesnenin parçasıyla karşılaşılırsa uzaklaştırılır. Köpeğin de ayağı vardır, masanın ve sandalyenin de. Köpek ayağı ile sandalye ayağı "ayak" olmaları bakımından benzer olsa da alakalı değildir. Model bunları birbirinden uzak tutar.

Bu yaklaşım hem metinde hem görüntüde kullanılır. Hoca cümle düzeyinde Word2Vec'in karşılığı olarak Sentence-BERT gibi modelleri andı. Eğitim pozitif ve negatif örnekler üzerinden yapılır. Rastgele negatifler yerine aynı belgeden alınan zor negatifler de kullanılır. Öğrenilen temsil, yeni bir örnek geldiğinde onun hangi gruba daha yakın olduğunu bulmak için kullanılır.

6.1 Fotoğraf Albümü Örneği

Aynı kişinin farklı fotoğrafları birbirine benzer olmalı, farklı kişilerin fotoğrafları birbirinden uzak olmalıdır. Telefonlardaki fotoğraf gruplayıcı uygulamalar bu mantıkla çalışır. Beş yıllık fotoğraflarınızı arka planda gruplar: Kendi yüzünüzün geçtiği fotoğraflar bir grup, doğa resimleri başka bir grup olur. Köpek fotoğrafları birbirine yaklaştırılır, kedi fotoğrafları köpeklerden uzaklaştırılır. Model böylece pozitif ve negatif örneklerden aynı nesnenin ne olduğunu öğrenir.

6.2 Kelime ve Cümle Düzeyinde Örnekler

- **Kelime düzeyi:** “Kedi” ile “tatlı” yakın, “kedi” ile “astronomi” uzak olur.
- **Cümle düzeyi:** “Hava çok sıcak” ile “Bugün kavurucu” yakın, “Hava çok sıcak” ile “Matematik zordur” uzak olur. “Matematik zordur” cümlesine ise “Bugün çalışmak çok zor” ya da “Bu sıcakta çalışmak zordur” gibi cümleler yakın bulunur.

6.3 Zor Negatifler

Rastgele negatifler kolaydır. Zor negatifler (hard negatives) ise benzer görünen ama anlamı farklı olan örneklerdir. Örneğin aynı belgenin farklı cümleleri benzer görünür ama anlamları genelde farklıdır. Hocanın örneği:

- “Kedi tatlı bir hayvandır.”
- “Kedi çok tehlikeli olabilir.”

İkisi de kediden bahseder ama anlamları zıttır. Modelin bunları ayırt etmesi için daha derin anlaması gerekir. Zor negatifler modele ince ayrımları öğretir.

7. İnsan Ödülü Olmadan Pekiştirmeli Öğrenme

Hoca geçen hafta anlattığı pekiştirmeli öğrenmeyi hatırlattı: Ödül önceden bir şekilde, örneğin insan tarafından etiketlenirse iyi sonuçlar alınabilir. Öğrenciler bunu Ödev 2’de yapmıştı. Soru şudur: Ödül insandan alınmadan pekiştirmeli öğrenme yapılabilir mi? Cevap evettir.

7.1 AlphaZero

Hoca öğrencilere AlphaZero’yu hatırlayıp hatırlamadıklarını sordu. Bir öğrenci “satranç robotu” diye yanıtladı. Hoca, ilk haftalarda oyunlar kısmında Deep Blue ile birlikte bundan bahsettiğini hatırlattı.

AlphaZero kendi kendine sürekli, belki milyonlarca oyun oynar. Oyunlardan hangi hamlelerin doğru, hangilerinin yanlış olduğunu görür. Doğrularını pekiştirir, yanlışlarını azaltır. Slayttaki basit Python koduna göre süreç şöyledir:

- Rastgele bir sinir ağı başlatılır.
- Oyunlardan örnekler toplanır.
- Bir sinir ağı eğitilir.
- Yeni ağ mevcuttakinden iyiyse kullanılır, kötüyse kullanılmaz.

Ödül bilgisi ortamdan, yani oyunun sonucundan gelir. Tamamen pekiştirmeli öğrenmedir ama insan etiketlemesi yoktur. Bu defalarca yapıldığı için satranç, dama gibi oyunlarda insana gerek kalmadan çok başarılı sonuçlar elde edilir.

Slayttaki şemada iki ajan vardır. A ajanında mevcut politika, B ajanında eski politika bulunur. İkisi birbiriyle oynar. A kazanırsa mevcut politika tutulur, B kazanırsa eski politika korunur. Politika bu şekilde sürekli iyileştirilir.

> **Ders sonrası not:** Altyazıda bu bölümde bir yerde “alpha-beta” geçiyor; bağlamdan büyük olasılıkla AlphaZero kastedilmiştir. AlphaZero’nun alfa-beta budamasıyla ilgisi yoktur; büyük olasılıkla kendi kendine oyunla eğitilen bir sinir ağı ve Monte Carlo ağaç araması kullanır.

7.2 Ödül Kaynakları

Slayta göre insan olmadan ödül üç yoldan gelebilir:

- **Kendi kendine oyun (self-play):** İki aynı program birbiriyle oynar, kazanan politika korunur. Satranç ve Go gibi.
- **Ortamdan gelen ödül:** Oyun skoru, hayatta kalma gibi sinyaller. Atari ve Mario gibi simülasyonlar.
- **Başka bir modelden gelen ödül:** İnsan geri bildirimli pekiştirmeli öğrenmenin (reinforcement learning from human feedback, RLHF) temelini oluşturur.

7.3 ChatGPT ve RLHF

Hoca ChatGPT'nin yalnız bir dil modeli olmadığını söyledi. Arka planda bir anlamda politikası güncellenir ve insanın istediği gibi cevap vermeyi öğrenir. Bunu geçen hafta anlattığını ama tekrar edeceğini belirtti. Aşamalar şunlardır:

1. **Eğiticili ince ayar (supervised fine-tuning):** İnsanlar örnek konuşmalar yazar, model bunları taklit eder. Hoca buradaki "küçük ölçekli" verinin 3-5 bin değil, belki yüz bin girdi-çıktı çifti olduğunu söyledi.
2. **Ödül modeli:** Model bir soruya birden çok cevap üretir, örneğin beş cevap. İnsan en iyisini seçer ya da sıralar. Öğrencilerin Ödev 2'de yaptığı gibi. Ödül modeli bu sıralamayı öğrenir.
3. **PPO ile pekiştirme:** PPO algoritmasıyla dil modeli, ödül modelini en yükseğe çıkaracak cevaplar üretmeyi öğrenir.
4. ve 3. adımlar defalarca, yinelemeli olarak tekrar eder. Hoca AlphaZero'da da aynı tekrar mantığının olduğunu belirtti.

Neden öz denetimli sayılıyor? Başta biraz insan etkisi vardır. Bir süre sonra insan devre dışı kalır ve makine kendi kendine öğrenir. Süreç makine üzerinden yürüdüğü için insan seviyesinin ötesine geçecek bir noktaya getirilebilir.

> **Ek bilgi (derste anlatılmadı):** PPO (Proximal Policy Optimization), OpenAI'ın büyük olasılıkla 2017'de yayımladığı genel amaçlı bir politika gradyanı algoritmasıdır. ChatGPT'ye özgü değildir, birçok pekiştirmeli öğrenme probleminde kullanılır. Temel fikri, her güncellemede politikanın eski politikadan çok uzaklaşmasını sınırlamaktır. RLHF'de bu, dil modelinin ödül modelini kandıracak tuhaf cevaplara kaymasını yavaşlatır.

8. Öz Denetimli Öğrenmenin Toplu Görünümü ve Ara Değerlendirme

Hoca anlatılanları model adları üzerinden toparladı. Slayttaki tabloya göre:

- **Üretici modeller:** otokodlayıcılar, VAE, difüzyon, GAN.
- **Tamamlayıcı modeller:** GPT, BERT, BART, ELECTRA.
- **Karşısal modeller:** SimCLR, MoCo, CLIP gibi modeller.
- **Pekiştirmeli modeller:** GPT'nin RLHF modülü, AlphaZero gibi.

Uygulamalar genelde birden çok yaklaşımı bir arada kullanır:

- ChatGPT: GPT ve pekiştirmeli öğrenme.
- Google aramaları: BERT.
- DALL-E: difüzyon ve CLIP.
- AlphaGo: kendi kendine oyun.
- Spotify önerileri: karşısal modeller.

Ardından hoca ekranda üç çoktan seçmeli soru gösterdi ve öğrencilerin cevaplarını aldı (Örnek Sorular bölümüne bakın). Sonunda öz denetimli öğrenmeyi şöyle özetledi: Verinin bir parçası gizlenir ya da bozulur ve model onu tahmin eder. Bu fikir otokodlayıcı, tamamlayıcı model, karşısal model ve pekiştirmeli öğrenme olarak farklı biçimlerde karşımıza çıkar.

8.1 2017 ve "Attention Is All You Need"

Hoca kronolojiye dikkat çekti. 2017 öncesinde CNN'lerin (evrişimli sinir ağları) öne çıktığı bir derin öğrenme furyası vardı. Makine öğrenmesi çalışmaları bir yere kadar gelip tıkanıyordu. Örneğin bir biyoinformatik veri setinde literatür başarısı %65 civarındaysa %70'e çıkılamıyor, hiperspektral görüntü işlemede %98 civarındaysa %99-100 yapılamıyordu. 2017'de yayımlanan "Attention Is All You Need" makalesi dikkat mekanizmasını ve Transformer yapısını getirdi. Sonraki birçok gelişme bunun üzerine kuruldu.

> **Hocanın vurgusu:** Bu alanda ya da mühendisliğin çeşitli alanlarında çalışacaksanız "Attention Is All You Need" makalesini okumanızı tavsiye ederim.

9. Üretici Modeller

Aradan sonra dersin ikinci konusuna geçildi. Üretici modeller, mevcut verilerin karakteristiğinden yola çıkarak yeni veriler üretir. Ya da sorulan soruya bir cevap üretir. Bu cevap metin, görüntü, ses ya da başka bir şey olabilir.

9.1 Dağılım Biliniyorsa

Etiketli ya da etiketsiz verilerin dağılımı biliniyorsa, yani verilerin ortalaması ve standart sapması (ya da varyansı) biliniyorsa, bu dağılımdan yeni örnekler üretilebilir. Hoca slaytta iki boyutlu iki veri bulutu gösterdi. Birinde varyans hem x hem y ekseninde büyük, diğerinde yalnız x ekseninde büyük, y ekseninde küçüktü. Dağılım biliniyorsa bu dağılımın içinden yeni örnekler üretilebilir.

Normal dağılım, yani Gauss eğrisi, bunun temel örneğidir. Eğrinin tepesi ortalamayı gösterir. Ortalamanın etrafındaki açıklığı standart sapma σ belirler. İki boyutlu veride iki boyutlu Gauss düşünülür.

Hocanın örneği (sayıların örnek amaçlı olduğunu kendisi belirtti): Bir toplumda ortalama boy $\mu = 170$ cm, standart sapma $\sigma = 5$ cm olsun. Bu durumda toplumun büyük bir kısmının boyu $\mu - \sigma = 165$ cm ile $\mu + \sigma = 175$ cm arasındadır. Hoca bu oranı "%65 ya da %67", bir yerde de "%67 ya da %70" olarak söyledi.

Hesap: Normal dağılımda $\mu \pm \sigma$ aralığında kalan oran

$$P(\mu - \sigma < X < \mu + \sigma) = \operatorname{erf}\left(\frac{1}{\sqrt{2}}\right) \approx 0{,}6827$$

yani yaklaşık %68,27'dir. Buna göre bu toplumda insanların yaklaşık %68'i 165-175 cm arasındadır.

> **Ders sonrası not:** Hoca bir standart sapma içindeki oranı "%65 veya %67" dedi; doğrusu %68,27'dir (Python ile doğrulandı).

Hedef, bu iki parametreden (ortalama ve standart sapma) yola çıkarak yeni veriler üretmektir. Üretilecek şey yalnız bir nokta değil, farklı türden veriler olabilir.

9.2 Dağılım Bilinmiyorsa: Otokodlayıcı ve VAE

Dağılım çoğu zaman bilinmez. O zaman otokodlayıcılarla örnek, sınıf, soru-cevap, özet gibi şeyler üretilebilir. Kodlayıcı, gizli uzay ve çözücü vardır. Girdi ile çıktı birbirine ne kadar yakınsa sonuç o kadar başarılıdır. Böylece örneğe benzer örnekler üretilebilir: kediye benzeyen kedi, köpeğe benzeyen köpek.

> **Hocanın vurgusu:** Veri sayısının az olduğu problemlerde bu çok önemlidir. Örneğin elinizde yeterli müzik yoksa, mevcut verilerden yeni müzik örnekleri üretebilirsiniz.

Sıkıştırma şöyle ifade edilir: $K \times K$ boyutlu bir görüntü (K satır, K sütun), $K \times K$'dan çok çok küçük d boyutlu bir vektöre indirgenir.

Klasik otokodlayıcıda gizli temsil sabit bir noktadır. VAE'de bir noktalar kümesine, bir bölgeye karşılık gelir. VAE temsil uzayını bir dağılıma sahip olacak şekilde düzenler. Bu sayede gürültüden yeni örnekler üretilebilir.

Slayttaki uygulamalar:

- MATLAB Deep Learning örneği: 20 boyutlu gizli uzayda çalışılıyor. Solda bir test örneği, sağda bu örneğin dağılımından üretilmiş örnekler var. Sıfıra benzeyen sıfırlar, dokuzlar üretilmiş. Bazıları biraz soru işareti uyandırsa da hoca iyi bir üretim olduğunu söyledi.
- Örnekler arası interpolasyon: 1'den 9'a kadar rakamlar arasında geçiş üretmek.
- Bir makalede kediden zürafaya interpolasyonla geçiş.

9.3 Günümüzün Üretici Modelleri ve Koşullu Üretim

Hoca günümüzde görüntü üreten (DALL-E, Midjourney), metin üreten (GPT-4, Llama, Gemini), ses ve video üreten modellerin artık sayılamayacak kadar çok olduğunu söyledi. Çoklu modlu (multimodal) modeller de vardır: metinden görüntü, görüntüden metin, metinden ses, sesten metin, görüntü ve metinden video.

Koşullu üretimde (conditional generation) model bir koşula göre üretim yapar. "Rastgele bir yüz üret" yerine "şöyle bir yüz üret" denir. Hoca algoritmaların arkasında ağır matematik olsa da kodlamanın kolay olduğunu, hazır fonksiyonlarla iki-üç satırda yazılabildiğini söyledi.

Koşullu üretim örnekleri:

- Hoca bazen bir yapay zeka aracından "bu metni bir proje hakemi üslubuyla yaz" diye istediğini anlattı.
- Slayttaki örnekte "Ahmet Hamdi Tanpınar üslubunda yaz" isteniyor. Model yazarın eserlerindeki dili, kelimeleri, kalıpları ve edebi unsurları öğrenip o üslupta yazmaya çalışır.
- Modlar arası dönüşüm: Whisper sesi metne çevirir. GitHub Copilot metinden kod üretir. Metinden resim, resimden metin de yapılabilir.

Hoca difüzyon modelini burada yeniden hatırlattı: Temizden gürültüye gidilir, sonra gürültüden temize dönülmeyi öğrenir.

9.4 GAN

Çekişmeli üretici ağlar (generative adversarial networks, GAN) iki modelin etkileşimiyle çalışır:

- **Üretici (generator):** Sahte (fake) örnek üretir.
- **Ayırıcı (discriminator):** Bir örneğin gerçek (real) mi sahte mi olduğunu tahmin eder. Elde gerçek veriler de vardır.

Hoca buradaki fikri "güzel bir şey" olarak niteledi: İkinci modelin başarısızlığı birinci modelin başarısıdır. Ayırıcı, üreticinin ürettiğini gerçeğinden ayırt edemiyorsa üretici başarılıdır. Buna çekişmeli ya da rekabetli eğitim (adversarial training) denir. Ayırıcının da rekabette geri kalmaması gerekir, bu yüzden o da eğitilir. Gerçek/sahte tahminleri üzerinden iki model sürekli kendini günceller. Bir noktadan sonra süreç insan etkisi olmadan ilerler.

> **Hocanın vurgusu:** Sınavda kaynakların içindeki kıyıda köşede kalmış ayrıntıları sormayacağız. Ama mühendis adayı biri için iş sınavdan ibaret değil. Özellikle henüz proje yapmamış arkadaşlar bu kaynakları incelemeli.

10. Transformer ve Öz Dikkat

Transformer mimarisi, RNN'lerden sonra "Attention Is All You Need" makalesiyle hayatımıza girdi.

Hocanın örneği: "Bankaya gittim, orada para yatırdım." "Orada" kelimesinin bankayı gösterdiğini biz anlarız. Transformer'daki dikkat mekanizması "orada"nın neye odaklanması gerektiğini bulur. Her kelime, cümledeki diğer hangi kelimelere dikkat etmesi gerektiğini öğrenir. Örneğin "gittim" için "nereye, nasıl gittim" sorularına cevap veren kelimelerle ilişki kurulur. Hoca bunu 5N1K'ye benzetti.

10.1 Adımlar

1. Kelimeler vektörlere dönüştürülür.
2. Konum bilgisi (positional encoding) eklenir. Klasik metin işlemede kelimeler doğrudan vektöre çevrilirdi. Ama kelimenin cümlenin birinci, ikinci ya da üçüncü öğesi olduğu bilgisi kaybolursa farklı bir anlam çıkabilir.
3. Öz dikkat (self-attention): Her token diğer token'larla ilişkilendirilir.
4. Token başına ileri beslemeli (feed-forward) dönüşüm uygulanır, ardından stabilizasyon gibi aşamalar gelir.

Hoca, kendisi doğal dil işleme alanında çalışmadığı için residual bağlantılar gibi kısımların teknik ayrıntısını bilmediğini, ne yaptıklarını bildiğini söyledi.

10.2 Karmaşıklık

Her token diğer tüm token'larla ilişkilendirildiği için çok büyük bir model ortaya çıkar. Hoca n kelime için $n(n-1)$, ikiye bölünürse $n(n-1)/2$ ilişki olduğunu, bunun da n^2 mertebesinde, yani $O(n^2)$ karmaşıklık demek olduğunu söyledi. Örneğin $n = 10$ için $n(n-1)/2 = 45$ çift vardır, $n^2 = 100$'dür. İkisi de $O(n^2)$ mertebesindedir.

10.3 Query, Key, Value

Basit öz dikkatte her token diğer token'larla nokta çarpımı (dot product) yapılarak benzerliğine bakılır. Benzerlikler normalize edilir, yani softmax fonksiyonundan geçirilir. Sonuçta bağlamın farkında olan gömme (embedding) vektörleri elde edilir. Hoca gömme vektörlerini bir anlamda özellik çıkarımıyla (feature extraction) elde edilmiş vektörler olarak tanımladı.

Esnek öz dikkatte her token üç farklı rolde bulunur:

- **Sorgu (query):** Ne arıyorum?
- **Anahtar (key):** Bende ne var?
- **Değer (value):** Bulunursam karşılığında ne veririm?

Hoca bunu kütüphanede arama yapmaya benzetti. Tekniklerine ve matematiğine girmeye gerek olmadığını, kabaca ne yaptığını bilmenin yeterli olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Transformer makalesindeki öz dikkat formülü $\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ biçimindedir. Burada QK^T tüm token çiftlerinin nokta çarpımlarını içeren $n \times n$ bir matristir; $O(n^2)$ maliyet buradan gelir. $\sqrt{d_k}$'ye bölmek, büyük boyutlarda nokta çarpımlarının çok büyüyüp softmax'ı doyurmasını önler.

10.4 İlk GPT'nin Yapısı

İlk GPT, ChatGPT'nin atası sayılabilir. Girdi metninin gömmeleri alınır, konum bilgisi eklenir, dropout (bazı nöronları devre dışı bırakma) uygulanır, Transformer bloklarından geçilir ve çıktı üretilir. Slayttaki şemaya göre her Transformer bloğu içinde şu sıra vardır: matris çarpımı, maskeleme (belli kısımların kapatılması), softmax, dropout ve yeniden matris çarpımı. Hoca bunların matematik olarak da, ne ve nasıl yaptıkları açısından da çok ayrıntılı olduğunu söyledi.

11. Büyük Dil Modelleri

Büyük dil modelleri (large language models, LLM) token tahminiyle eğitilir. Günümüzde otoregresif modeller standart hale gelmiştir ve yeni görevlere kolay uyum sağlar. Slaytta "LLM'ler sıfır hataya erişebilir mi?" diye bir soru vardı. Hoca vektör temsillerinin nasıl olacağı gibi konuların hâlâ araştırıldığını söyledi. Bir LLM'e "Bir varmış" dediğinizde "bir yokmuş" diye devam eden üretimler yapabilir.

Bu modeller bu işleri yaparken dilin mantığını, gerçekleri ve matematiği öğrenmek zorundadır. Hoca slayta göre GPT-4'ün çok büyük bir parametre sayısına ulaştığını söyledi.

11.1 Ölçek ve Faz Geçişi

Hoca, sudan buza beklenmedik faz geçişinden esinlenen bir fikri anlattı. Sıcaklık çok az düşer ve davranış bir anda tamamen değişir. Büyük dil modelleri de belli bir ölçeği geçince yepyeni yetenekler gösterir. Altyapı kurulur ve küçük bir dokunuşla kod yazma, mantık yürütme gibi yetenekler ortaya çıkar. Ölçek arttıkça yetenekler artar ama doğrusal artmaz. Slayta göre GPT-3 matematikte zayıfken GPT-4 güçlüdür. Hoca bunun tam olarak nasıl sağlandığını bilmediğimizi ekledi.

11.2 Zero-Shot, One-Shot, Few-Shot

Hoca RLHF'de başta 100.000 gibi bir veriyle eğitim yapılıp sonrasının otomatikleştirildiğini hatırlattı. Bunun da ötesinde x-shot öğrenme yöntemleri önerilmiştir:

- **Zero-shot:** Hiç örnek verilmez. "İngilizceden Türkçeye çevir" denir ve model çevirir.
- **One-shot:** Tek bir örnek verilir: "şunu şuna çevir" gibi.
- **Few-shot:** Birkaç örnek verilir.

Geleneksel makine öğrenmesinde her görev için ayrı model eğitilir: OCR sınıflandırma için bir model, e-posta sınıflandırma için başka bir model. LLM ise yeni görevleri yalnız prompt ile anında yapabilir. Gradyan inişi (gradient descent) gibi bir güncelleme yoktur, birkaç prompt ile sonuç gösterir.

Slayttaki bir uygulamada çoktan seçmeli sorularda x-shot kullanılıyor: Her cevap seçeneğinin üretilme olasılığı hesaplanır ve en yüksek olasılıklı seçenek seçilir. Örnek soru "What blocks sunshine?" ve seçenekler "summer, park, desktop, sea, moon" idi. Hoca bir de "Öksürdüm, çünkü …" örneğini verdi: Devam olarak "sigara içtim", "sesim kısıldı" gibi seçeneklerin her birinin olasılığı vardır ve en yükseği alınır.

11.3 InstructGPT

Hoca InstructGPT'yi RLHF uygulayan ve ChatGPT'nin atası sayılabilecek model olarak gösterdi ve makalesine işaret etti. Aşamalar yine aynıdır: önce eğiticili öğrenme, sonra ödül modeli öğrenmesi, sonra ikisinin tekrarı. Model 1'in çıktısı ödül modeline (model 2) sunulur, model 1 model 2'nin ödül tahminini kullanarak pekiştirmeli öğrenmeyle eğitilir, sonra 2. adıma dönülür. Hoca bugünkü ChatGPT'nin muhtemelen artık daha farklı bir yöntem kullandığını, yayımlananların sınırlı olduğunu söyledi.

Hoca ayrıca büyük dil modellerini taklit eden daha küçük modeller olduğunu söyledi. Birkaç gün önce çok büyük bir dil modelinin 16 MB'a kadar sıkıştırıldığını gösteren bir video gördüğünü ve paylaşacağını belirtti.

12. Halüsinasyon, İnce Ayar ve RAG

12.1 Halüsinasyon

Hoca dil modellerinin bir açmazı olduğunu söyledi: Bir soruya çok ciddi bir cevap verebilir ama bu cevap yanlış olabilir. Öğrencilere sordu:

Soru: Bir dil modelinden yanlış cevap aldığınız oldu mu? Ya da aldığınız cevabın doğru olduğuna kanaat getirip sorgulamadan kullandınız mı? **Cevap:** Bir öğrenci, Türkçe bir dil modeli olan Cosmos'un bir fizik sorusunda yer çekimini dörde bölmesi gereken yerde bunu yapamadığını, yanlış sonuç verdiğini söyledi. Hoca kendisi de "Türkiye'nin başkenti Paris" gibi bir cevap örneğini verdi.

Dil modellerinde yeni bilgiyi anında öğretme imkanı yoktur. Bu yüzden halüsinasyon (hallucination) oluşabilir. Modeli "bak, o öyle değil, aslı böyle" diye düzeltmek için ince ayar (fine-tuning) gerekir. Bu çok zaman alır, süreç tekrar tekrar baştan alınır.

Hoca halüsinasyona bir slayt örneği de gösterdi: Cosmos GPT'ye insanlık tarihindeki en büyük icatlar sorulmuş. Model "1899'da geliştirilen insansız araba", "47'de yapay zeka sistemleri", "2012'de Alan Turing" gibi cevaplar vermiş. Hoca modelin yılları büyük olasılıkla karıştırdığını söyledi. Bunlar gerçek gibi görünen ama yanlış bilgilerdir.

12.2 İnce Ayar ile RAG Benzetmesi

Hoca farkı bir sınav benzetmesiyle anlattı:

- **İnce ayar:** Sınava çalışarak girmek. Yeterince soru çözülür, tam ezber olmasa da ezbere benzer. Ama yanlış bir şey ezberlenmişse, bir şey gözden kaçmışsa ya da soru ters köşe olursa yanlış cevap verilebilir.
- **RAG:** Sınava kitapla (referans kitapla) girmek. Her soru için doğru kaynak bulunur.

Bilgi getirmeyle güçlendirilmiş üretimin (retrieval-augmented generation, RAG) avantajları:

- Anlık, güncel bilgiye erişim. LLM'in eğitiminden sonra olan haberler modelde yoktur, RAG'da bu sorun kalkar.
- Kuruma özel bilgiye erişim. Yerelde çalıştırılıp kurum belgelerinden cevap üretilebilir.
- Kaynak gösterme. "Bu cevabı şu belgeden ürettim" denir.
- Halüsinasyonu azaltma. Sıfırlamak belki mümkün değildir ama azaltılır.
- Model yeniden eğitilmez.

Hoca RAG'ı açıklanabilir yapay zekaya (explainable AI) yakın gördü: Model ürettiği cevabın kaynağını da söyler. "Kitap olmadan cevap yok."

12.3 Bölüm Yönetmelikleri Projesi

Hoca geçen dönem yaptırdıkları bir RAG projesini anlattı. Bölümün staj yönetmeliği, proje yönetmeliği ve öğrenci genel yönetmeliği vardır. Amaç, öğrencinin bir soruyu sekreterliğe sormadan önce bir grafik arayüzden sorabilmesiydi. GPT gibi genel bir modele "staj kaç gündür?" diye sorulduğunda bazen "25 gün" gibi yanlış cevaplar gelebiliyordu. RAG ise ilgili kavramla ilişkili belgelere bakarak, bağlama göre ve kaynağıyla cevap getiriyordu.

Hoca bunun Ctrl+F gibi bir anahtar kelime araması olmadığını, bağlama göre arama olduğunu, ama ortada bir model eğitimi de bulunmadığını vurguladı.

12.4 Üslü Sayılar Örneği

Hoca RAG tabanlı bir çalışmadan bahsetti: İlkokul öğrencilerine matematik öğreten bir yapay zeka sistemi. Sisteme ders kitabı verilir, sistem konuyu anlatır ve sorular sorar. Hocanın örneği üslü sayılardı. Kitapta iki kural yazar:

$$x^a \cdot x^b = x^{a+b}, \qquad y^a \cdot z^a = (y \cdot z)^a$$

Bu iki kuraldan yola çıkarak sistem, örneğin $6^5 \cdot 2^3 \cdot 3^3$ gibi bir ifadeyi önce ikinci kuralla $2^3 \cdot 3^3 = 6^3$ yapıp, sonra birinci kuralla $6^5 \cdot 6^3 = 6^8$ hesaplayabilecek yeteneğe ulaşabilir. Yani iki kuralı birleştirerek kitapta doğrudan yazmayan bir soruyu çözebilir (sayısal çözüm Örnek Sorular bölümünde).

12.5 RAG Nasıl Çalışır?

RAG iki aşamalıdır:

1. **İndeksleme (çevrim dışı, offline):** Veri tabanı ya da belgeler yüklenir. Belgeler küçük parçalara (chunk) bölünür. Parçalar gömme (embedding) modeliyle sayısal vektörlere çevrilir. Vektörler bir vektör veri tabanına kaydedilir.
2. **Sorgulama ve üretim:** Soru da vektöre çevrilir. Bu vektöre en yakın k vektör, yani k parça bulunur. Bu parçalardan bir bağlam oluşturulur ve LLM'e verilir. LLM soruyu bu bağlama göre cevaplar. Cevapla kaynak eşleştirilir: "Bu cevabı şu şu parçalardan verdim."

Hocanın örneği: "2024 yılı Ar-Ge bütçesi nedir?" Bu bilgi bir yerde vardır. Belki birebir "Ar-Ge bütçesi" diye yazmaz ama yakın bir ifade geçer. Sistem bununla ilgili k parçayı alır ve LLM yardımıyla yanıtı verir.

12.6 Parçalama (Chunking)

Parçalama sabit boyutlu, örtüşmeli, anlamsal (semantic) ya da belge yapısına göre olabilir. Slayttaki ayar: parça boyutu (chunk size) 512, örtüşme (overlap) 50. Kelime bazında düşünülürse sistem kaynaktan 512 kelime alıp vektörleştirir. Sonraki parça, bu 512 kelimenin son 50 kelimesiyle başlar ve 512 kelime daha alır. Hoca örtüşmeli parçalamanın özellikle teknik belgelerde kullanıldığını söyledi.

Hoca derste "$512 - 50 = 462$, 462'den başlayıp bir 512 daha alıyorsun" dedi. İlk parça 1-512. kelimelerdir. Son 50 kelime 463-512. kelimelerdir. Dolayısıyla ikinci parça 463. kelimeden başlar ve $462 + 512 = 974$. kelimede biter (çözüm Örnek Sorular bölümünde).

Ders sonrası not: Hoca ikinci parçanın "462'den başladığını" söyledi; doğrusu ikinci parça 462. kelimeden sonra, yani 463. kelimeden başlar.

12.7 Gömme ve Arama

Gömme modelleri bir metni vektöre dönüştürür. Hoca OpenAI'ın "text-embedding-3" modelini örnek gösterdi. Öğrenciler de Ödev 2'de bazı gömme yöntemleri kullanmıştı. Bugün bu yöntemlerden binlercesi vardır. Vektörler bir vektör veri tabanına konur ve arama yapılır. Arama, veri yapıları tarzında arama yapılarıyla ve anlamsal (semantic) olarak yapılır. Örneğin bir soru için sistem, kod numarası tam tutmasa da anlamca yakın olduğu için "bilgi güvenliği politikası" belgesini döndürebilir.

Slayttaki örnekte ilk 20 parça getirilip dil modeline soruluyor. Hatta bazı sistemler birden fazla dil modeline sorup gelen cevaplardan en uygununu döndürür. Hoca ileri RAG tekniklerine dair kodlama içeren kaynakları paylaşacağını söyledi.

Ek bilgi (derste anlatılmadı): RAG'da "en yakın k vektör" genellikle kosinüs benzerliği ya da iç çarpım ile ölçülür. Kosinüs benzerliği iki vektör arasındaki açının kosinüsüdür: $\cos\theta = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$. Değer 1'e yaklaştıkça iki metin anlamca daha yakın kabul edilir. Büyük veri tabanlarında tüm vektörlerle tek tek karşılaştırmak yerine büyük olasılıkla yaklaşık en yakın komşu (approximate nearest neighbor) indeksleri kullanılır.

12.8 LLM'lerin Sınırları: Plan Yapma

Hoca, slaytın hazırlandığı dönemde bazı dil modellerinin plan yapmada başarısız kalabildiğini söyledi. Geçen dönem bölümde proje sınavlarının jüri programını dil modellerine yaptırmayı denemişler. Klasik genetik algoritma tarzı yöntemler ve diğer endüstri mühendisliği teknikleri daha başarılı çıkmış. Dil modelleri kısmen plan yapabiliyor ama tam değil.

Hoca son olarak Fatih Hoca'nın grubunun dil modellerini kodları ve açıklamalarıyla anlatan bir video serisi olduğunu, sınav için değil ama ilgilenenler için inceleyebileceklerini söyledi.

13. Colab Uygulaması: Otokodlayıcı ile MNIST Sınıflandırma

Dersin sonunda hoca başta yazdırdığı kodu Google Colab'de çalıştırıp sonuçları gösterdi. Google Colab'de bir Python kodunun rahatça yazılıp deneysel sonuçlar üretilebildiğini söyledi.

Kodun akışı:

- MNIST verisi yüklenir ve bir data loader oluşturulur.
- Otokodlayıcı tanımlanır. İleri (forward) aşamada önce kodlayıcı, sonra çözücü çalışır. GPU varsa kullanılır.
- Eğitimden sonra her görüntü için 32 değer ve 33. sütun olarak etiket (0-9) içeren bir veri tablosu oluşur.
- Değişkenlere bakılınca 10.000 örnek olduğu, her birinin 32 değer içerdiği görüldü. Orijinal veri 10.000 × 784'tür.
- Veri 5 katlı çapraz doğrulamayla 5 kez eğitim/test olarak bölünür ve 5 kez test edilir. Doğruluklar (accuracy) ortalanır.
- Sınıflandırıcılar: KNN ($k = 5$), SVM (RBF çekirdekli) ve MLP. Hoca KNN'de $k = 1$ seçmenin daha güzel olabileceğini, ama 5'in de sorun olmadığını söyledi. SVM'de RBF dışında başka çekirdekler denenebileceğini ekledi.

Sonuçlar:

Deney	Boyut	Ortalama doğruluk
Otokodlayıcı + KNN/SVM/MLP	32	yaklaşık %92-94
Ham veri + KNN/SVM/MLP	784	yaklaşık %94-95

Otokodlayıcı 15 epoch'ta kaybı yaklaşık 0,01'e indirdi. Ham veriyle yapılan deney yaklaşık 128 saniye sürdü, yani daha uzun sürdü. Ancak otokodlayıcının eğitim süresi de hesaba katılırsa toplam süre belki ilk deneyde daha fazladır.

Hocanın yorumu: Bu basit problemde otokodlayıcı başarıyı artırmadı, hatta ham veri bazı durumlarda daha iyi çıktı. Ham veriyle çalışmak ise daha uzun sürdü.

> **Hocanın vurgusu:** Bu basit problem için otokodlayıcı anlamlı olmayabilir. Problem zorlaştıkça, veri çok yüksek boyutlu oldukça ve özellikle üretim yaptırılacaksa otokodlayıcılar ve VAE'ler başarımı iyileştirir.

Hoca otokodlayıcının arkasında çok matematik olsa da basit bir otokodlayıcının ilgili kütüphanelerle 15-20 satırlık bir kod olduğunu, yazmasının zor olmadığını söyledi.

Örnek Sorular

Bu derste video ve kare kullanılmadı. Aşağıdaki sorular derste sözlü sorulan ya da hocanın okuyup cevapladığı sorulardır.

1. (derste sözlü, 00:37:24) Word2Vec bağlamında: "kedi" kelimesinin etrafında olabilecek kavramlar nelerdir?

Çözüm (derste verildi): Kedi ile birlikte kullanılan "mama", "tatlı", "güzel", "oyuncu" gibi kelimeler vektör uzayında kediye yakın olur. "Bisiklet tekerleği", "astronomi" gibi alakasız kavramlar uzakta kalır.

2. (derste sözlü, 00:49:53) AlphaZero neydi?

Çözüm (derste verildi): Bir öğrenci "satranç robotu" dedi, hoca onayladı. AlphaZero kendi kendine milyonlarca oyun oynayarak, ödülü insandan değil oyunun sonucundan alarak öğrenir.

3. (derste sözlü, 00:57:05) Ekranda gösterilen 1. çoktan seçmeli soru: GPT ile BERT arasındaki temel fark nedir?

Çözüm (derste verildi): Doğru şık B: BERT iki yönlü bağlam kullanır, GPT soldan sağa ilerler.

4. (derste sözlü, 00:58:09) Ekranda gösterilen 2. soru: Difüzyon modelleri nasıl öğrenir?

Çözüm (derste verildi): Doğru şık C: Model ileri yönde eklenen gürültüyü öğrenir, sonra gürültüyü adım adım geri temizler.

5. (derste sözlü, 00:58:32) Ekranda gösterilen 3. soru: Karşısal öğrenmede zor negatifler (hard negatives) neden kullanılır?

Çözüm (derste verildi): Temsillerin ince detayları yakalayabilmesi için. Zor negatifler benzer görünen ama anlamı farklı örnekleri ayırmayı öğretir, temsilin kaba ve genel kalmasını önler.

6. (derste sözlü, 01:42:01) Kitapta yalnız $x^a \cdot x^b = x^{a+b}$ ve $y^a \cdot z^a = (yz)^a$ kuralları varken, RAG tabanlı bir sistem $6^5 \cdot 2^3 \cdot 3^3$ ifadesini nasıl hesaplar?

Çözüm (derste verildi, ara adımlar Python ile doğrulandı):

- İkinci kural: $2^3 \cdot 3^3 = (2 \cdot 3)^3 = 6^3$. Kontrol: $8 \cdot 27 = 216 = 6^3$.
- Birinci kural: $6^5 \cdot 6^3 = 6^{5+3} = 6^8$.
- Sonuç: $6^8 = 1.679.616$ (ara adım eklendi).

7. (derste sözlü, 01:44:37) Parça boyutu 512 kelime, örtüşme 50 kelime ise ikinci parça hangi kelimeden başlar?

Çözüm: Hoca "$512 - 50 = 462$ mi? 462'den başlayıp bir 512 daha alıyorsun" dedi.

- İlk parça: 1-512. kelimeler.
- İlk parçanın son 50 kelimesi: $512 - 50 + 1 = 463$. kelimeden 512. kelimeye kadar.
- İkinci parça 463. kelimeden başlar ve $463 + 512 - 1 = 974$. kelimede biter (ara adım eklendi, Python ile doğrulandı).

  Ders sonrası not: 462, ilk parçada örtüşmeden önce kalan kelime sayısıdır. İkinci parça 462. kelimeden değil, 463. kelimeden başlar.

Dersin Toparlaması

Bu derste eğiticili öğrenmenin en büyük darboğazından, yani yeterli etiketli veri bulmanın zorluğundan yola çıkıldı. Öz denetimli öğrenme bu soruna, cevabı verinin kendisinden üreterek çözüm getirir: Verinin bir parçası maskelenir, silinir ya da bozulur ve model onu tahmin eder. Bu fikrin dört biçimi görüldü: otokodlayıcılar (ve onların geliştirilmiş hali VAE ve difüzyon modelleri), tamamlayıcılar (GPT, BERT, Word2Vec, ELECTRA, BART), karşısal öğrenme (pozitifleri yaklaştırma, negatifleri uzaklaştırma, zor negatifler) ve insan ödülü olmadan pekiştirmeli öğrenme (AlphaZero, RLHF ve PPO). MNIST üzerindeki Colab deneyi, basit problemlerde boyut indirgemenin başarıyı her zaman artırmadığını gösterdi.

İkinci bölümde veri de az olduğunda üretici modellerin nasıl yeni örnek ürettiği anlatıldı: dağılım biliniyorsa doğrudan örnekleme, bilinmiyorsa VAE, GAN ve difüzyon. Ardından bugünkü büyük dil modellerinin temeli olan Transformer ve öz dikkat, ölçekle ortaya çıkan zero/one/few-shot yetenekleri, halüsinasyon sorunu ve buna karşı ince ayar ile RAG yaklaşımları karşılaştırıldı. Hoca makine öğrenmesi konularının bu derste bittiğini, son konu olan “Yapay Zeka ve Etik”in bir sonraki derste işleneceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.